CORE103 Computational Thinking
PW9 - Nested Lists



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions . . . 2
1.2 Objectives . . . 2
1.3 Use of AI . . . 2

2 Prerequisites **3**
2.1 Assert Statements . . . 3
2.2 Pytest . . . 3

3 Exercises **5**
3.1 Compute My Bill . . . 5
3.2 Bill Bill . . . 6
3.3 Muhammad Huzaifa XI – Part II . . . 8
3.4 Special List Sort . . . 9
3.5 The Collatz Family . . . 11
3.6 Seniority . . . 13
3.7 Heroes in a Grid . . . 15
3.8 Getting (A Matrix) Into Shape! . . . 17
3.9 Keyboard . . . 19
3.10 Programming Challenge - Snakes and Ladders . . . 21

Introduction

1.1 Instructions

- This PW is **UNGRADED**. However, it is highly encouraged that you attempt and submit it to get feedback from your section tutor.
- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW9_aa123456.zip` where `aa123456` will be replaced with your student id.

1.2 Objectives

Following are the objectives of this practice worksheet:

- Understand the concept of nested lists in Python and their role in organizing complex data structures.
- Explore techniques for accessing and manipulating elements within nested lists.
- Apply nested lists to solve programming challenges that involve multi-dimensional data, demonstrating proficiency in nested list manipulation and problem-solving.

1.3 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
========================================================================================== test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

========================================================================================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts =============================
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

================================== FAILURES ===================================
_____________________________ test_question1[lst10-lst20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ___
```

Exercises

3.1 Compute My Bill

This question should be implemented in the file ***q1.py****.*

Problem Description

A grocery store records purchased items in a nested list. Each inner list contains the item name, unit price, and quantity in the following format:

[item_name, unit_price, quantity]

The store needs an itemized receipt showing the total cost of each item, the total quantity purchased, and the final bill.

For each item, multiply its unit price by its quantity and append the result to its inner list in the receipt. At the end, add a summary list in the following format:

['Quantity', total_quantity, 'Total Bill', total_bill]

Function Description

Write a function **compute_my_bill** that takes purchased_items, a nested list containing item details, and **returns** a new nested list representing the receipt.

Constraints

- Each item contains a non-empty name, a non-negative price, and a positive integer quantity.
- Prices have at most two decimal places.
- Round calculated costs to two decimal places.
- Preserve the original order of items.
- Do not modify the original nested list.
- Taxes and discounts are not considered.
- The input list may be empty.

Sample Interaction

```
>>> purchased_items = [
...     ['Milk', 10.99, 2],
...     ['Cake', 12.50, 5],
...     ['Tapal Family Mixture', 450, 1],
...     ['Shan Bombay Biryani', 70, 2]
... ]

>>> compute_my_bill(purchased_items)
[['Milk', 10.99, 2, 21.98],
 ['Cake', 12.5, 5, 62.5],
 ['Tapal Family Mixture', 450, 1, 450],
 ['Shan Bombay Biryani', 70, 2, 140],
 ['Quantity', 10, 'Total Bill', 674.48]]

>>> compute_my_bill([])
[['Quantity', 0, 'Total Bill', 0]]
```

Sample Interaction Explanation

In the first example, the total quantity is $2 + 5 + 1 + 2 = 10$, and the final bill is $21.98 + 62.50 + 450 + 140 = 674.48$.

In the second example, no items are purchased, so both totals are zero.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

3.2 Bill Bill

This question should be implemented in the file ***q2.py****.*

Problem Description

A sushi restaurant keeps a record of customers' visits. Each visit is stored as a list containing the customer's tab ID, followed by the prices of items ordered.

For example, **[12, 2.50, 5.00, 3.50]** represents three charges for customer 12.

A customer may visit the restaurant multiple times. Your task is to calculate the total bill for each visit made by a given customer.

Function Description

Write a function **my_bill** that takes all_charges, a nested list of customer visits, and tab_id, the customer's ID.

The function **returns** a list of bill totals for the given customer, in the order of their visits. Return an empty list if the customer has no recorded visits.

Constraints

- Each inner list contains an integer tab ID, followed by zero or more float charges.
- Negative charges represent refunds or adjustments.
- A visit with no charges has a total of zero.
- Round each bill total to two decimal places.
- Do not modify the original nested list.

Sample Interaction

```
>>> all_charges = [
...     [12, 2.32, 5.60, 2.29, 6.08, 7.12],
...     [35, 18.29, 18.30, 5.55, 5.55],
...     [4, 12.00],
...     [35, 3.0],
...     [7, 3.95, 4.95, 5.95, 6.95],
...     [7, 5.20, -2.00],
...     [12, 0.0],
...     [7, 13.69, 2.49]
... ]

>>> my_bill(all_charges, 4)
[12.0]

>>> my_bill(all_charges, 7)
[21.8, 3.2, 16.18]

>>> my_bill(all_charges, 420)
[]

>>> my_bill([[8]], 8)
[0]
```

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Muhammad Huzaifa XI – Part II

This question should be implemented in the file ***q3.py****.*



Problem Description

Muhammad Huzaifa wants to organize the registered players into three categories based on the vowels in their names:

- **Category A**: Names that start **and** end with a vowel.
- **Category B**: Names that start **or** end with a vowel, but not both.
- **Category C**: Names containing at least one vowel, but neither starting nor ending with a vowel.

Names without any vowels are excluded from all three categories.

Function Description

Write a function **dostiyaariXI** that takes player_names, a list of player names, and **returns** a nested list containing three lists in the order: Category A, Category B, and Category C.

Each category must contain its player names sorted using Python's default alphabetical ordering.

Constraints

- Each player name is a non-empty string.
- Vowels are **a, e, i, o, u**, regardless of case.
- A name may contain uppercase letters, lowercase letters, digits, or symbols.
- Do not modify the original player_names list.
- An empty input returns [[], [], []].

Sample Interaction

```
>>> player_names = [
...     'quddusi', 'swifthyderabad', 'CEmajorAT',
...     'skibidiali', 'valverde', 'baqir',
...     'ronaldo', 'hassan', 'alonso', 'etoo',
...     'mrgreedy', 'mbappe', 'messi', 'evra',
...     'mraura', 'amir', 'mufaddal', 'isco',
...     'saifibhai', 'iniesta', 'abdullah',
...     'hussainshah', 'askarizz', 'basim',
...     'raphina', 'usman', 'coldp@lm4r'
... ]

>>> dostiyaariXI(player_names)
[
    ['alonso', 'etoo', 'evra', 'iniesta', 'isco'],
    ['abdullah', 'amir', 'askarizz', 'mbappe',
     'messi', 'mraura', 'quddusi', 'raphina',
     'ronaldo', 'saifibhai', 'skibidiali',
     'usman', 'valverde'],
    ['CEmajorAT', 'baqir', 'basim', 'coldp@lm4r',
     'hassan', 'hussainshah', 'mrgreedy',
     'mufaddal', 'swifthyderabad']
]

>>> dostiyaariXI(
...     ['Ava', 'Amir', 'Bilal', 'O']
... )
[['Ava', 'O'], ['Amir'], ['Bilal']]

>>> dostiyaariXI([])
[[], [], []]
```

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

3.4 Special List Sort

This question should be implemented in the file ***q4.py****.*

Problem Description

A university stores student records in a nested list. Each inner list contains a student's name, major, and batch year in the following format:

[student_name, major, batch_year]

The university wants to sort these records using the following order of priority:

1. Batch year (ascending)
2. Major (alphabetical order)
3. Student name (alphabetical order)

Function Description

Write a function **special_list_sort** that takes student_records, a nested list containing student details, and **returns** a new nested list sorted according to the given priorities.

Constraints

- Each inner list contains exactly three values: student name (string), major (string), and batch year (integer).
- Batch years are four-digit integers.
- Student names use title case, and major codes use uppercase letters.
- Preserve all student records without modifying the original nested list.
- The input list may be empty.

Sample Interaction

```
>>> student_records = [
...     ['Sarwan', 'EE', 2021],
...     ['Ayesha Khan', 'CND', 2021],
...     ['Hamza Junaid', 'EE', 2021],
...     ['Ahsan Qadeer', 'CS', 2020],
...     ['Muhammad Ali Bhutto', 'EE', 2020],
...     ['Marium Habiby', 'SDP', 2021],
...     ['Adil Ali Khan', 'EE', 2021]
... ]

>>> special_list_sort(student_records)
[['Ahsan Qadeer', 'CS', 2020],
 ['Muhammad Ali Bhutto', 'EE', 2020],
 ['Ayesha Khan', 'CND', 2021],
 ['Adil Ali Khan', 'EE', 2021],
 ['Hamza Junaid', 'EE', 2021],
 ['Sarwan', 'EE', 2021],
 ['Marium Habiby', 'SDP', 2021]]

>>> special_list_sort([])
[]
```

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q4.py
```

3.5 The Collatz Family

This question should be implemented in the file ***q5.py****.*

Problem Description

The Collatz conjecture generates a sequence starting from a positive integer x using the following rules:

- If x is even, divide it by 2.
- If x is odd, replace it with $3x + 1$.

The Collatz family extends the odd-number rule to $3x + 2n + 1$, where n is an integer called the *family number*. Each family has specific cycle numbers, as shown below.

Family (n)	**Cycle Numbers**
-2	0, 3, 15, 51
-1	1, 5, 17
0	1
1	3
2	1, 5, 19, 23, 187

Your task is to generate the sequence for each integer in a given range. For each sequence, **apply the rules at least once**, then continue until a cycle number for the selected family is reached. Even if the starting number is already a cycle number, it must undergo at least one step before the sequence can stop.

Function Description

Write a function **collatz_cousin** that takes three integers: n, start, and stop.
The function must **print** the sequence for every integer from start to stop, inclusive, in increasing order of starting numbers.
Each sequence must include its starting number and the first cycle number reached **after at least one step**, following the format shown in the sample interactions.

Constraints

- All three parameters are integers.
- $-2 \leq n \leq 2$
- $1 \leq$ `start` $\leq$ `stop`
- Use integer division when x is even.

- A sequence must take at least one step, even if its starting number is a cycle number.
- You may assume that each sequence in the given test cases reaches one of the listed cycle numbers.

Sample Interactions

```
>>> collatz_cousin(0, 1, 20)
```

```
1, 4, 2, 1
2, 1
3, 10, 5, 16, 8, 4, 2, 1
4, 2, 1
5, 16, 8, 4, 2, 1
6, 3, 10, 5, 16, 8, 4, 2, 1
7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
8, 4, 2, 1
9, 28, 14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
10, 5, 16, 8, 4, 2, 1
11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
12, 6, 3, 10, 5, 16, 8, 4, 2, 1
13, 40, 20, 10, 5, 16, 8, 4, 2, 1
14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
15, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1
16, 8, 4, 2, 1
17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
18, 9, 28, 14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
20, 10, 5, 16, 8, 4, 2, 1
```

```
>>> collatz_cousin(2, 41, 60)
```

```
41, 128, 64, 32, 16, 8, 4, 2, 1
42, 21, 68, 34, 17, 56, 28, 14, 7, 26, 13, 44, 22, 11, 38, 19
43, 134, 67, 206, 103, 314, 157, 476, 238, 119, 362, 181, 548, 274, 137, 416, 208, 104, 52, 26, 13, 44, 22, 11, 38, 19
44, 22, 11, 38, 19
45, 140, 70, 35, 110, 55, 170, 85, 260, 130, 65, 200, 100, 50, 25, 80, 40, 20, 10, 5
46, 23
47, 146, 73, 224, 112, 56, 28, 14, 7, 26, 13, 44, 22, 11, 38, 19
48, 24, 12, 6, 3, 14, 7, 26, 13, 44, 22, 11, 38, 19
49, 152, 76, 38, 19
50, 25, 80, 40, 20, 10, 5
51, 158, 79, 242, 121, 368, 184, 92, 46, 23
52, 26, 13, 44, 22, 11, 38, 19
53, 164, 82, 41, 128, 64, 32, 16, 8, 4, 2, 1
54, 27, 86, 43, 134, 67, 206, 103, 314, 157, 476, 238, 119, 362, 181, 548, 274, 137, 416, 208, 104, 52, 26, 13, 44, 22, 11, 38, 19
55, 170, 85, 260, 130, 65, 200, 100, 50, 25, 80, 40, 20, 10, 5
56, 28, 14, 7, 26, 13, 44, 22, 11, 38, 19
57, 176, 88, 44, 22, 11, 38, 19
58, 29, 92, 46, 23
59, 182, 91, 278, 139, 422, 211, 638, 319, 962, 481, 1448, 724, 362, 181, 548, 274, 137, 416, 208, 104, 52, 26, 13, 44, 22, 11, 38, 19
60, 30, 15, 50, 25, 80, 40, 20, 10, 5
```

Sample Interaction Explanation

In the first example, $n = 0$. For each starting number from 1 to 20, the sequence follows the standard Collatz rules. It stops when 1 is reached **after at least one step**. Therefore, even starting from 1 produces:

$$1 \to 4 \to 2 \to 1$$

For example, starting from 5:

$$5 \to 16 \to 8 \to 4 \to 2 \to 1$$

In the second example, $n = 2$. For each starting number from 41 to 60, the odd-number rule becomes $3x + 5$.
For example, starting from 58:

$$58 \to 29 \to 92 \to 46 \to 23$$

The sequence stops at 23 because it is a cycle number for $n = 2$.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```

3.6 Seniority

This question should be implemented in the file ***q6.py****.*

Problem Description

A university club wants to arrange its members from youngest to oldest. It stores their names and ages in two separate lists, where elements at the same index belong to the same person.

Your task is to sort the names by age in ascending order. If two members have the same age, preserve their original order.

Function Description

Implement the following functions:

- **seniority**: Takes two lists, names and ages, and **returns** the names sorted by age in ascending order.
- **main**: Takes no parameters. Reads two lines of comma-separated input, prepares a list of names (strings) and a list of ages (integers), passes both lists to **seniority**, and **prints** the returned list.

Input Format

The input consists of two lines:

- First line: Comma-separated names.
- Second line: Comma-separated ages.

Constraints

- Both lists have the same non-zero length.
- All names are non-empty strings.
- All ages are positive integers.
- Names may contain spaces.
- Do not modify the original names or ages lists.

Sample Interaction

In the samples below, lines written as **[Input -> value]** represent user input, not program output.

```
>>> main()
[Input -> waqar, jamal, umair, shahid]
[Input -> 20, 22, 19, 18]
['shahid', 'umair', 'waqar', 'jamal']
```

```
>>> main()
[Input -> Ayesha, Bilal, Sara, Hamza, Zoya, Omar]
[Input -> 22, 25, 21, 20, 22, 21]
['Hamza', 'Sara', 'Omar', 'Ayesha', 'Zoya', 'Bilal']
```

Sample Interaction Explanation

In the first example, Shahid is the youngest at 18, followed by Umair (19), Waqar (20), and Jamal (22).

In the second example, Sara and Omar are both 21, while Ayesha and Zoya are both 22. Their original relative order is preserved.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q6.py
```

3.7 Heroes in a Grid

This question should be implemented in the file ***q7.py****.*

Problem Description

Two heroes are lost in a grid of rooms and are trying to find each other. Each room contains either:

- **0**: An empty room.
- **1**: A room occupied by a hero.

There are exactly two heroes in the grid.

A hero can move one room at a time: up, down, left, or right, but not diagonally.

Your task is to find the minimum number of steps needed for one hero to reach the other. This is called the **rectangular distance**.

Function Description

Implement the following functions:

- **rectangular_distance**: Takes grid, a nested list of integers, and **returns** the rectangular distance between the two heroes.
- **main**: Takes no parameters. Reads the input, creates the nested list grid, passes it to **rectangular_distance**, and **prints** the returned distance.

Input Format

- The first line contains two space-separated integers, n and m, representing the number of rows and columns.
- The next n lines each contain exactly m digits (0 or 1), without spaces or commas.

The function **main** must convert the input into a nested list of integers.

Constraints

- $2 \leq n, m \leq 400$
- The grid contains exactly two **1**s.
- All other cells contain **0**.
- The function must not modify the original grid.

Sample Interaction

In the samples below, lines written as **[Input -> value]** represent user input, not program output.

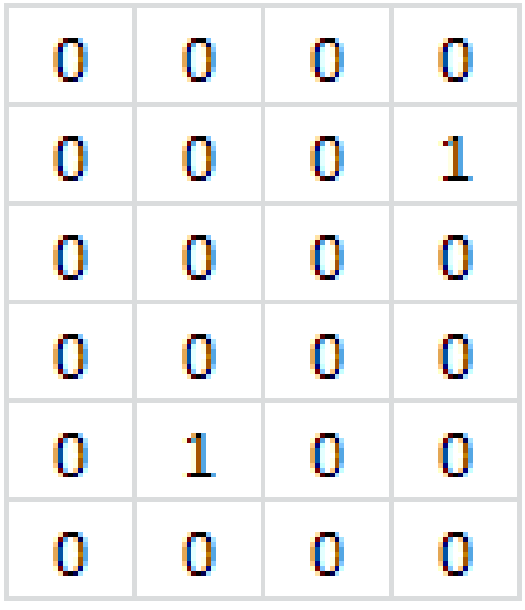
```
>>> main()
[Input -> 6 4]
[Input -> 0000]
[Input -> 0001]
[Input -> 0000]
[Input -> 0000]
[Input -> 0100]
[Input -> 0000]
5
```

```
>>> main()
[Input -> 5 5]
[Input -> 10000]
[Input -> 00000]
[Input -> 00000]
[Input -> 00000]
[Input -> 00001]
8
```

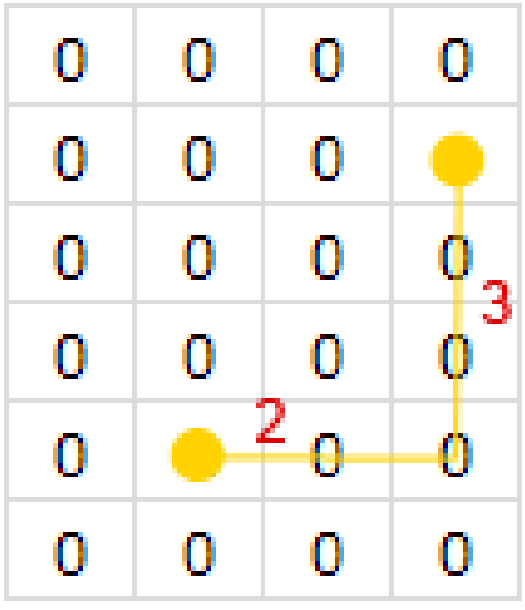
```
>>> main()
[Input -> 2 2]
[Input -> 01]
[Input -> 01]
1
```

Sample Interaction Explanation

In the first example, the heroes are at positions $(1, 3)$ and $(4, 1)$, using 0-based indexing. The first figure shows their positions, while the second shows the distance between them.



Original Grid



Rectangular Distance

The heroes are 3 rows and 2 columns apart. Therefore, the rectangular distance is $3 + 2 = 5$ steps.

In the second example, the heroes are at opposite corners of a 5×5 grid. The distance is $4 + 4 = 8$ steps.

In the third example, the heroes are in neighboring rooms, so the distance is 1 step.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q7.py
```

3.8 Getting (A Matrix) Into Shape!

This question should be implemented in the file ***q8.py****.*

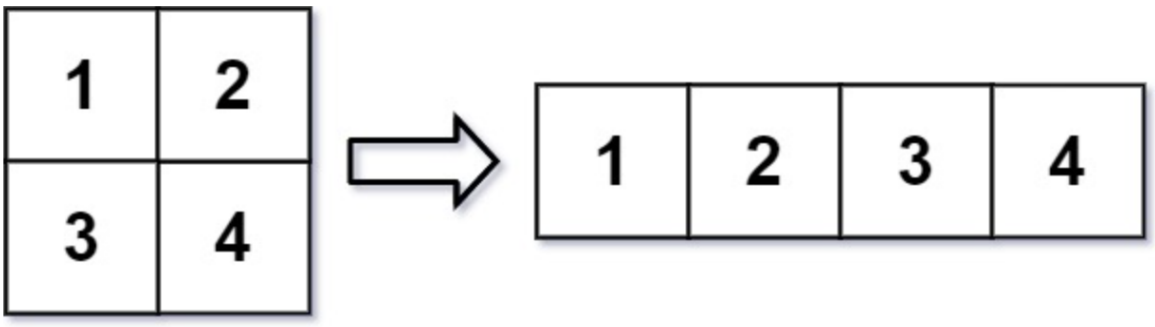
Problem Description

A matrix can be reshaped into a different number of rows and columns while preserving its elements.

Given an $m \times n$ matrix, your task is to reshape it into an $r \times c$ matrix while keeping the elements in their original row-by-row order.

Reshaping is possible only if $m \times n = r \times c$. Otherwise, the original matrix must be returned.

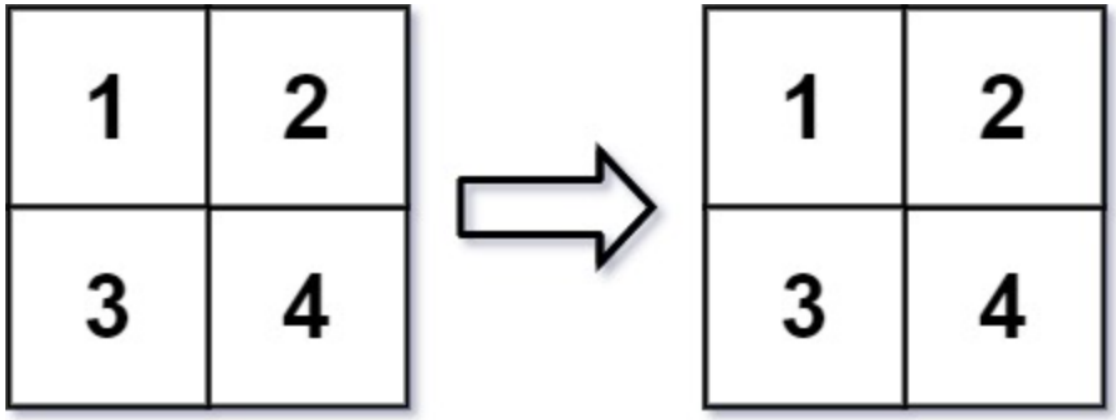
Example 1:




```
Input: mat = [[1,2],[3,4]], r = 1, c = 4
Output: [[1,2,3,4]]
```

Explanation: The original matrix has 4 elements. Since the requested shape is 1×4, reshaping is possible.

Example 2:




```
Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]
```

Explanation: The requested shape is 2×4, which requires 8 elements. Since the original matrix contains only 4 elements, the original matrix is returned.

Function Description

Implement the following functions:

- **matrixReshape**: Takes mat, a nested list of integers, and two integers r and c. It **returns** the reshaped matrix, or the original matrix if reshaping is not possible.
- **main**: Takes no parameters. Reads the input, constructs the nested list mat, and passes mat, r, and c to **matrixReshape**. It then **prints** the returned matrix.

Input Format

- The first line contains four space-separated integers: m, n, r, and c.
- The next m lines each contain n comma-separated integers representing one row of the original matrix.

Constraints

- $1 \leq m, n \leq 100$
- $-1000 \leq$ `mat[i][j]` ≤ 1000
- $1 \leq r, c \leq 300$
- The input matrix is rectangular.
- The function must not modify the original mat nested list.

Sample Interaction

In the samples below, lines written as **[Input -> value]** represent user input, not program output.

```
>>> main()
[Input -> 2 2 1 4]
[Input -> 1,2]
[Input -> 3,4]
[[1, 2, 3, 4]]
```

```
>>> main()
[Input -> 2 2 2 4]
[Input -> 1,2]
[Input -> 3,4]
[[1, 2], [3, 4]]
```

```
>>> main()
[Input -> 2 3 3 2]
[Input -> 1,2,3]
[Input -> 4,5,6]
[[1, 2], [3, 4], [5, 6]]
```

Sample Interaction Explanation

In the first example, the original matrix contains $2 \times 2 = 4$ elements. The requested shape is 1×4, so reshaping is possible. All elements are placed in a single row.

In the second example, the requested shape requires $2 \times 4 = 8$ elements, but the original matrix contains only 4. Therefore, the original matrix is returned.

In the third example, both the original and requested shapes contain 6 elements. The matrix is reshaped from 2×3 to 3×2, preserving the original row-by-row order.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q8.py
```

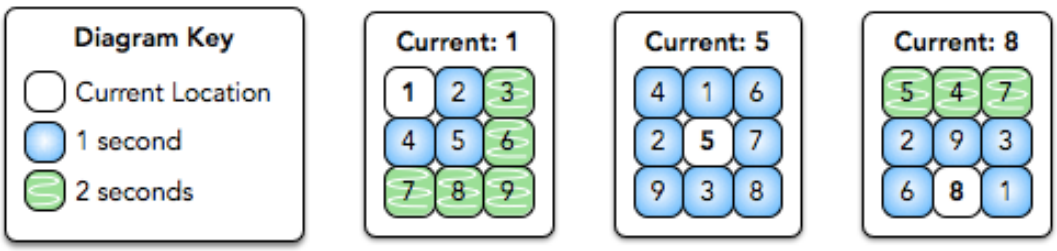
3.9 Keyboard

*This question should be implemented in the file **q9.py**.*

Problem Description

A secure facility changes the arrangement of its 3×3 numeric keypad every day. Your task is to calculate the minimum time needed to type a sequence of digits using the following rules:

- Moving to the first key takes 0 seconds.
- Pressing the current key takes 0 seconds, even when pressed repeatedly.
- Moving to an adjacent key horizontally, vertically, or diagonally takes 1 second.
- Moving to a non-adjacent key requires multiple moves between adjacent keys.



This diagram depicts the minimum amount of time it takes to move from the current location to all other locations on the keypad.

Function Description

Write a function **entryTime** that takes digits_to_type, a string of digits, and keypad, a string representing the keypad layout.

Every three consecutive digits in keypad represent one row of the keypad, from top to bottom.

The function **returns** the minimum total time, in seconds, needed to type the given sequence.

Constraints

- keypad contains exactly 9 distinct digits.
- digits_to_type is a non-empty string containing only digits present in keypad.
- Each move to an adjacent key costs 1 second.

Sample Interaction

```
>>> entryTime('423692', '923857614')
8

>>> entryTime('444', '923857614')
0
```

Sample Interaction Explanation

In the first example, the keypad layout is:

9	2	3
8	5	7
6	1	4

The time required for each movement is:

- Start at 4: 0 seconds.
- 4 to 2: 2 seconds.
- 2 to 3: 1 second.
- 3 to 6: 2 seconds.
- 6 to 9: 2 seconds.
- 9 to 2: 1 second.

The total time is $0 + 2 + 1 + 2 + 2 + 1 = 8$ seconds.

In the second example, the same key is pressed three times, so no movement is required. The total time is 0 seconds.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q9.py
```

3.10 Programming Challenge - Snakes and Ladders

This question should be implemented in the file ***q10.py****.*

Problem Description

You are given an $n \times n$ nested list board representing a Snakes and Ladders game.

The squares are numbered from 1 to n^2. Numbering starts at the bottom-left corner (**board[n-1][0]**), moves from left to right, and then from right to left in the next row. This pattern continues until the top row.

Each cell contains either:

- **-1**: No snake or ladder.
- **[row, column]**: The position where a snake or ladder takes the player.

The player starts at square 1. For each number in diceRolls, the player moves forward by that many squares. Only the square where the player lands is checked for a snake or ladder.

If the player lands on a snake or ladder, move to the position given in that cell. If the new position also has a snake or ladder, follow it as well. Continue until the player reaches a normal square.

All given dice rolls are valid: every roll moves the player to a square within the board. You do not need to handle a roll that would go beyond square n^2. Process **every** roll in diceRolls and record the player's position after each turn. The player does not have to reach the last square.

Function Description

Write a function **playSandL** that takes board, the game board, and diceRolls, a list of dice results.

The function **returns** a nested list containing the player's position after each dice roll, in order. Each position must be a list in the format **[row, column]**, using 0-based indices.

The returned list must contain exactly **len(diceRolls)** positions.

Constraints

- $n \geq 2$ and n is even. The board contains n rows and n columns.
- Each dice roll is an integer from 1 to 6.
- Each dice roll results in a valid move within squares 1 to n^2.
- Every roll is processed. If square n^2 is reached, it is reached on the final roll.
- Each cell contains **-1** or a valid position **[row, column]**.
- Squares 1 and n^2 have no snake or ladder.
- Any snake-and-ladder chain encountered during a turn eventually reaches a normal square.
- Do not modify the original board or diceRolls lists.

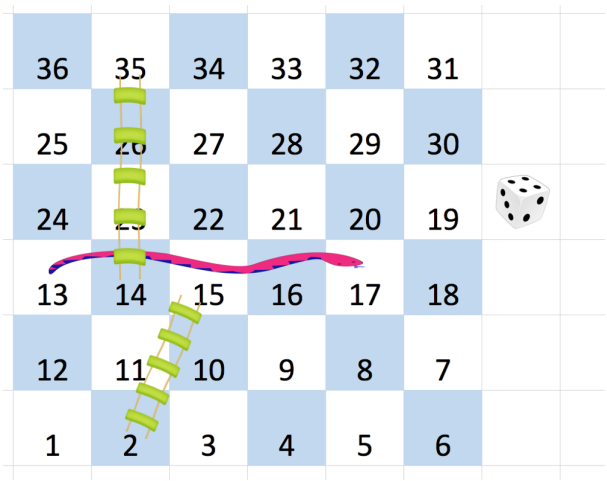
Sample Interaction 1

```
>>> board = [
...     [-1, -1, -1, -1, -1, -1],
...     [-1, -1, -1, -1, -1, -1],
...     [-1, -1, -1, -1, -1, -1],
...     [-1, [0, 1], -1, -1, [3, 0], -1],
...     [-1, -1, -1, -1, -1, -1],
...     [-1, [3, 2], -1, -1, -1, -1]
... ]

>>> diceRolls = [1, 2, 1, 1]

>>> playSandL(board, diceRolls)
[[3, 2], [3, 0], [0, 1], [0, 0]]
```

Sample Interaction 1 Explanation



Starting Position	Dice Roll	Movement and Final Position
[5, 0]	1	[5, 1] → [3, 2] (Ladder)
[3, 2]	2	[3, 4] → [3, 0] (Snake)
[3, 0]	1	[3, 1] → [0, 1] (Ladder)
[0, 1]	1	[0, 0] (Normal move)

The final position after each turn is added to the returned list.

Sample Interaction 2

```
>>> board = [
...     [-1, [3, 4], -1, [5, 5], -1, -1],
...     [-1, [2, 1], -1, [0, 5], -1, -1],
...     [-1, [0, 1], -1, [1, 3], -1, -1],
...     [-1, [1, 5], [2, 0], -1, [3, 0], -1],
...     [-1, [4, 2], -1, [3, 3], [1, 1], -1],
...     [-1, -1, [1, 2], [3, 2], -1, -1]
... ]

>>> diceRolls = [1, 6, 6, 2, 1, 2]

>>> playSandL(board, diceRolls)
[[5, 1], [3, 0], [2, 5], [0, 5], [0, 4], [0, 2]]
```

Sample Interaction 2 Explanation

Starting Position	Dice Roll	Movement and Final Position
[5, 0]	1	[5, 1] (Normal move)
[5, 1]	6	[4, 4] → [1, 1] → [2, 1] → [0, 1] → [3, 4] → [3, 0]
[3, 0]	6	[2, 5] (Normal move)
[2, 5]	2	[2, 3] → [1, 3] → [0, 5]
[0, 5]	1	[0, 4] (Normal move)
[0, 4]	2	[0, 2] (Normal move)

Some turns involve more than one snake or ladder.

The player must follow all of them before the turn ends. In this example, the dice rolls are exhausted at [0, 2], so the player does not have to reach [0, 0].

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q10.py
```